



Solving the Lorenz ODE System using Optimal ANN Architectures

Can a **plain feedforward ANN** — no physics in the loss — learn a coupled nonlinear ODE system, and **which architecture does it best?**

Ikramul Hasan Morad

Md. Abu Bakar

Samiur Rahman Omlan

Fariha Islam

Md. Touhidul Islam

Supervisor **Dr. Muhammad Nomani Kabir**, Professor, Department of CSE

Introduction & Motivation

Coupled nonlinear ODEs like the **Lorenz-1960** system — an early three-equation meteorological model — rarely admit closed-form solutions. Classical Runge–Kutta solvers are accurate but **mesh-dependent**, re-integrating step-by-step for every query. A trained network can instead learn the continuous map $\mathbf{t} \rightarrow (\mathbf{x}, \mathbf{y}, \mathbf{z})$ in one shot — yet most neural-ODE work leans on physics-informed losses or operator learning.

Central question: can a **plain feedforward ANN** — trained only on numerical data, with no physics in the loss — approximate the Lorenz-1960 solution, and **which architecture does it best?**

Objectives

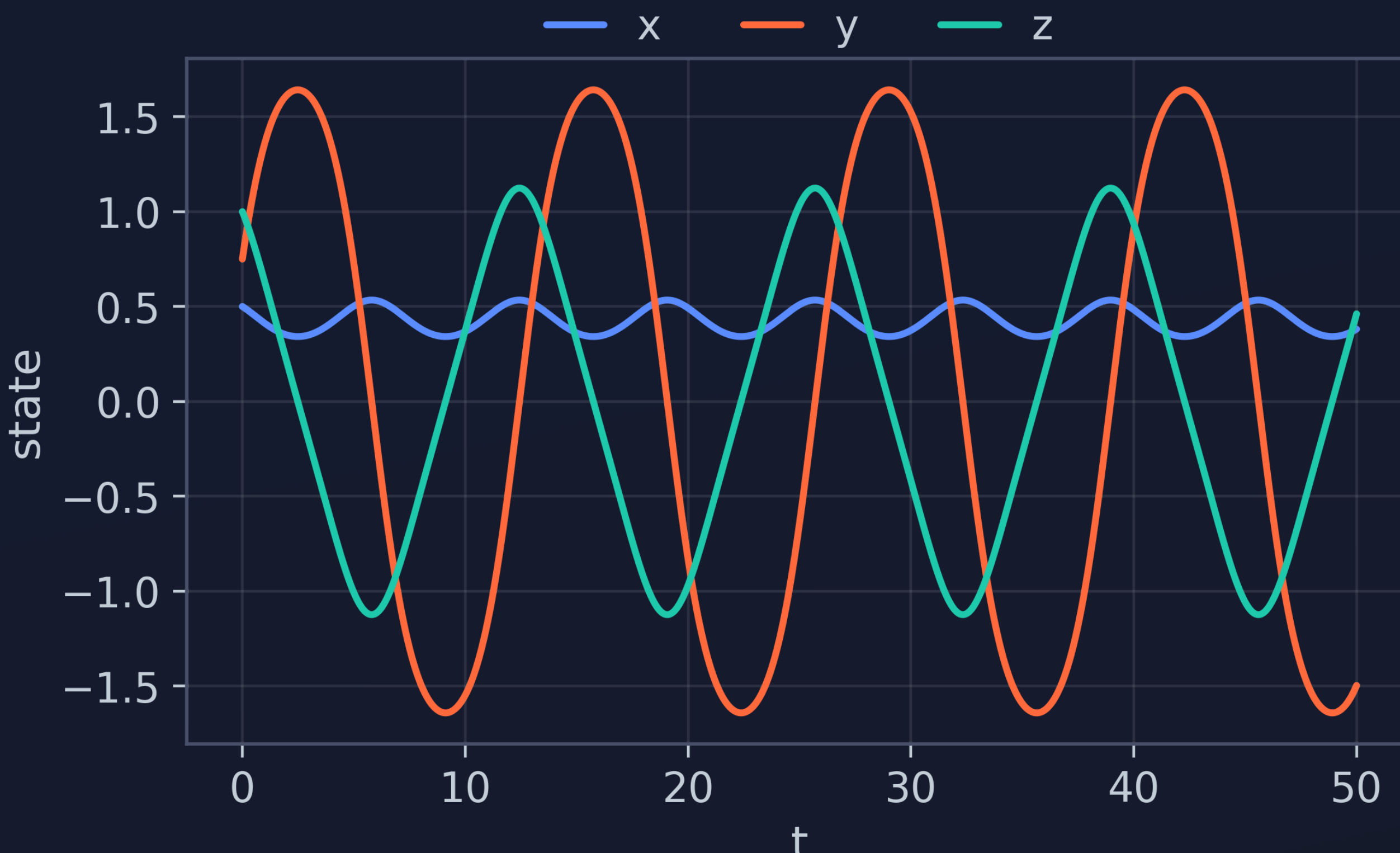
- Build a trusted reference with **RK4** and **SciPy**; cross-validate the two solvers.
- Systematically vary **depth**, **width**, and **activation** — plain ANN, no physics aids.
- Score accuracy vs cost: MSE, MAE, max error, training / inference time.
- Benchmark the best plain ANN against **PINN** / **DeepONet** results from the literature.

Background: The System

$$\begin{aligned}\dot{x} &= -0.10 \, y \, z \\ \dot{y} &= +1.60 \, x \, z \\ \dot{z} &= -0.75 \, x \, y\end{aligned}$$

$k = 2, \ell = 1 \quad \cdot \quad x_0=0.5, \, y_0=0.75, \, z_0=1 \quad \cdot \quad t \in [0, 1]$

Three coupled nonlinear ODEs — compact, but a genuinely demanding benchmark for any solver.



Bounded, quasi-periodic orbits over $t \in [0, 50]$.

Research Gap & Questions

Gap. No prior work runs a systematic plain-ANN architecture study on Lorenz-1960; existing studies fix the architecture, use only simple ODEs, or rely on physics-informed losses.

What's missing

- No architecture study for Lorenz-1960
- No PINN vs plain-ANN comparison on it
- Depth **and** width rarely varied together

Research questions

- Can a plain ANN learn the solution from numerical data alone?
- Which depth $\times$ width $\times$ activation wins the accuracy–cost trade-off?
- How close does it come to physics-informed baselines?

Methodology — Two-Phase Study

69 controlled runs

Phase 1 — Architecture search

depth {1, 2, 3, 4} $\times$ width {20, 50, 100} $\times$ activation {tanh, ReLU, sigmoid, GELU, Swish}

= 60 runs $\cdot$ Adam fixed

Phase 2 — Optimizer comparison

3 best Phase-1 architectures $\times$ {Adam, L-BFGS, SGD + momentum}, everything else held constant.

= 9 runs

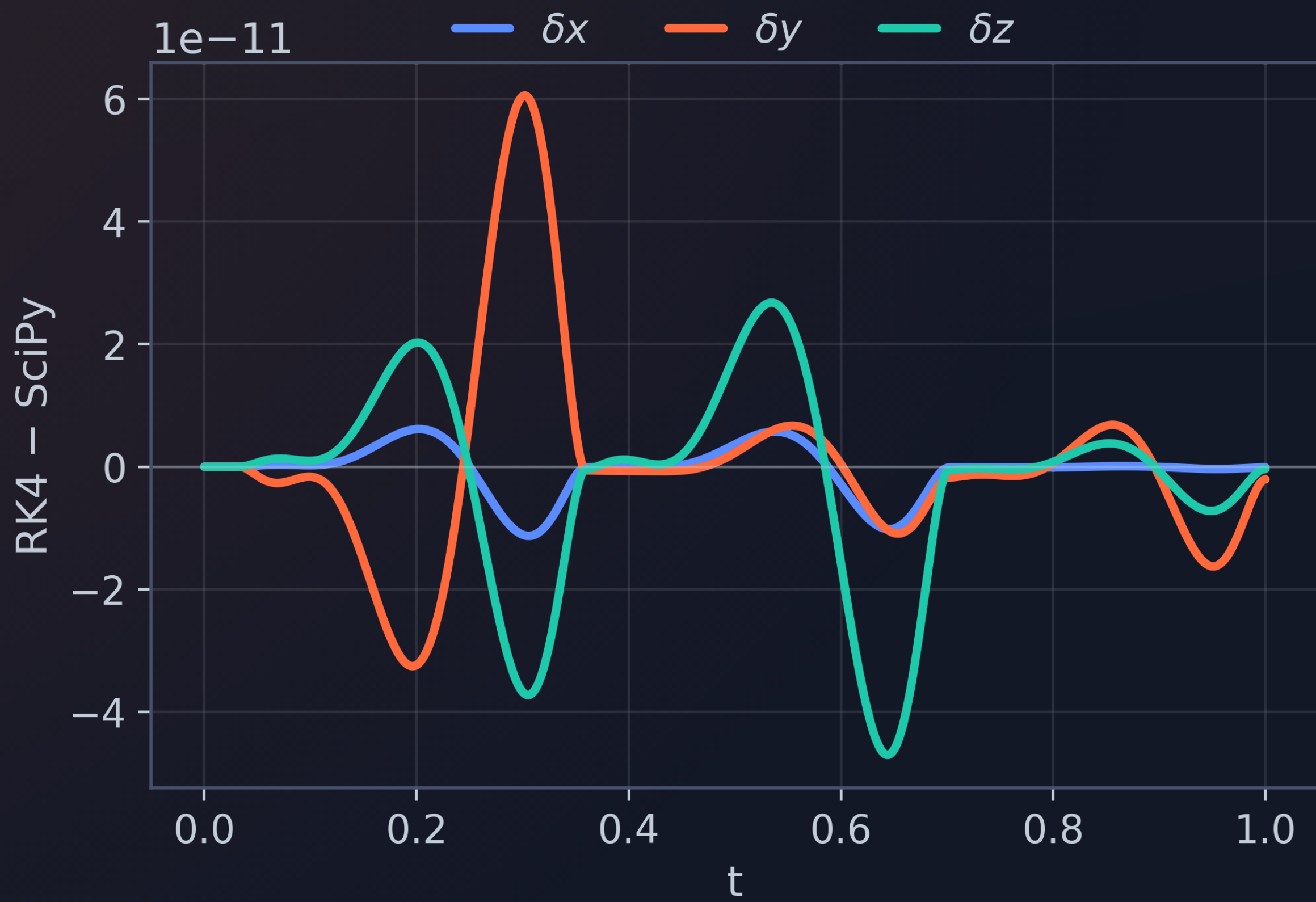
69 controlled experiments — splitting the phases cleanly separates **architecture** effects from **optimizer** effects.

Generate
RK4 + SciPyPreprocess
80/20 splitTrain
ANN gridEvaluate
5 metricsSelect
best

Numerical Baseline & Validation

Result

The reference trajectory is produced by two independent solvers and cross-checked **before any ANN sees it**: a custom **RK4** integrator ($h = 10^{-3}$, 1001 points) and **SciPy DOP853** ($\text{rtol} = 10^{-10}$), split 80/20 for train–validation (seed 42).



Pointwise RK4 - SciPy disagreement stays at the 10^{-11} level across the trajectory.

1.8×10^{-22}

solver-agreement MSE (RK4 vs SciPy)

✓ target was $< 10^{-10}$

A defensible **ground truth** for all FYDP-II ANN training.

Status (FYDP-I) & Future Work

Delivered this stage

- Research question framed; honest, narrow scope
- 17-paper review with an explicit gap
- Two-phase design (69 runs) locked
- Dual-solver baseline verified

Next — FYDP-II

- Build the training pipeline; run Phase 1 (60), then Phase 2 (9)
- Repeat seeds for stability; denser grids & new initial conditions
- Compare directly with PINN / operator-learning baselines

No accuracy claims are made until the FYDP-II experiments produce evidence — the foundation is set, the results come next.

Selected References

[1] Lorenz (1960), *Tellus* 12(3).

[2] Matthews & Bihio (2024), *PinnDE*.

[3] Raissi et al. (2019), *JCP* 378.

[4] Lagaris et al. (1998), *IEEE TNN* 9(5).

[5] Lau & Werth (2020), *ODEN*.

[6] Chen et al. (2018), Neural ODEs, *NeurIPS*.

Team Lorenz-1960 $\times$ ANN

Final Year Design Project I

Dept. of CSE, UIU · June 2026

github.com/ihmorad/fydp_workspace